

Project Report — GLPI · n8n Dashboard

Author: Mohamed Bakkali Affiliation: Faculté des Sciences de Meknès Employer: CGI.ca Date: 2026-05-20

Executive Summary

This project assembles a locally runnable, Docker Compose–based environment that demonstrates automated GLPI ticket creation from spoken audio using n8n workflows and Groq APIs, and provides observability via Prometheus, Grafana and Loki. The repository includes infrastructure compose files, exported n8n workflows, monitoring provisioning, a small Go sample app used as a test target, scripts for log/metric collection, and comprehensive documentation.

This report consolidates the repository contents, architecture, deployment steps, configuration, operational guidance, monitoring, security considerations, and artifacts useful for handoff or portfolio presentation.

Goals & Scope

- Demonstrate an audio → AI → ITSM automation pipeline: browser/app audio → n8n webhook → Groq Whisper (transcription) → Groq LLM (extraction) → GLPI ticket creation.
- Provide a reproducible, local dev environment using Docker Compose for GLPI, n8n, Prometheus, Grafana, Loki, Promtail and a sample Go app.
- Offer monitoring dashboards and log aggregation for troubleshooting and capacity observation.
- Ship clear documentation and sanitized configuration templates for safe public sharing.

Scope excludes: production hardening, scaling to Kubernetes, or provisioning cloud resources.

High-level Architecture

- User audio (browser or voice interface) → n8n webhook
- n8n nodes convert audio, call Groq Whisper for transcription, call Groq LLM to extract ticket metadata
- n8n authenticates to GLPI REST API and creates a ticket
- n8n sends logs to Loki; Prometheus scrapes metrics from services (n8n, sample app, cadvisor)
- Grafana reads Prometheus and Loki to provide dashboards

The environment is implemented as Docker Compose stacks (infra/tutorial-environment-docker-compose.yml for monitoring + n8n + app; infra/glpi-docker-compose.yml for GLPI + MySQL).

Components

- GLPI: PHP-based ITSM application (containerized). Compose file: `infra/glpi-docker-compose.yml`.
- MySQL: Database backing GLPI (named volumes for persistence).
- n8n: Workflow engine hosting the audio→LLM→GLPI flows. Workflows exported in `n8n/workflows/workflows.json` and documented in `n8n/workflows/README.md`.
- Groq APIs: external Whisper (speech-to-text) and LLM services used by n8n nodes (API keys provided by operator via environment/n8n credentials).
- Prometheus: metrics collection (configured in `infra/prometheus/prometheus.yml`).

- Grafana: dashboards and provisioning under ``infra/grafana/`` (dashboards JSON, provisioning YAML).
- Loki + Promtail: log aggregation and shipping (promtail config in ``infra/promtail/config.yml``).
- Dashboard app: small Go app in ``dashboard/app/`` used as a target for monitoring and logs simulation.
- Scripts: utilities under ``scripts/`` (e.g., ``groq_to_loki.py``) used to demonstrate pushing metrics/logs.

Key Files / Artifacts

- ``README.md`` — repo entry point and index
- ``docs/`` — full documentation (00–10): project overview, architecture, installation, configuration, workflows, GLPI integration, dashboard, monitoring, Docker services, troubleshooting, security
- ``infra/glpi-docker-compose.yml`` — GLPI + MySQL compose
- ``infra/tutorial-environment-docker-compose.yml`` — n8n + Prometheus + Grafana + Loki + sample app
- ``n8n/workflows/workflows.json`` — exported n8n workflows (no embedded credentials)
- ``dashboard/app/`` — Go sample app source and Dockerfile
- ``docs/screenshots/`` — UI screenshots for documentation
- ``.env.example`` — sanitized environment template

n8n Workflows (summary)

The repository contains two primary exported workflows (see ``n8n/workflows/workflows.json``):

1. Demo/workflow (inactive) — simple audio → transcribe → GLPI ticket flow. 2. "Click & Speak ITSM" (active) — production-style flow with error handling and Loki logging. Steps: - Receive multipart audio via webhook (``/webhook/nouveau-ticket``) - Convert audio (if needed) - Call Groq Whisper for transcription - Call Groq LLM to extract ticket fields (name, description, urgency, device, location, etc.) - Authenticate to GLPI and create a ticket via REST API - Emit logs to Loki for auditing

Configuration notes:

- Credentials are stored in n8n's credential store (encryption key controlled via ``N8N_ENCRYPTION_KEY`` in ``.env``).
- Exports do not include credentials; users must create credentials in n8n UI after import.

GLPI Integration

- GLPI is accessed via its REST API (e.g., ``/Ticket`` endpoints). The workflows use HTTP nodes to authenticate and post ticket payloads.
- Demo credentials in documentation: ``glpi:glpi`` (for development/demo only). For production, use a dedicated API user and secure credentials in n8n.

Monitoring & Observability

- Prometheus scrapes services configured in ``infra/prometheus/prometheus.yml`` (jobs include: `tns_app`, `n8n`, `cadvisor`).
- Grafana is pre-provisioned with dashboards under ``infra/grafana/dashboards/``:
- ``docker-monitoring.json`` — container CPU/memory/IO monitors
- ``groq-api-stats.json`` — Groq API usage
- ``n8n-glpi-monitor.json`` — workflow metrics

- Loki receives application and workflow logs via Promtail configuration; Grafana dashboards query Loki for log-based troubleshooting.

Dashboard App (sample)

The ``dashboard/app/`` folder contains a small Go application that exposes an instrumented ``/metrics`` endpoint and a simple web UI. It's used as a test target for the monitoring stack and as a source of synthetic traffic and logs (a ``loki/web-server-logs-simulator.py`` script helps generate logs).

Docker & Deployment

Two primary compose stacks:

- GLPI stack: ``infra/glpi-docker-compose.yml`` (GLPI + MySQL)
- Monitoring / workflow stack: ``infra/tutorial-environment-docker-compose.yml`` (n8n, prometheus, grafana, loki, promtail, cadvisor, sample app)

Basic local run (development):

```
# Start monitoring and workflow stack (from repo root)
docker-compose -f infra/tutorial-environment-docker-compose.yml up -d
```

```
# Start GLPI stack
docker-compose -f infra/glpi-docker-compose.yml up -d
```

Notes:

- ``.env`` values are read by Compose; copy ``.env.example`` to ``.env`` and fill values before starting.
- Volumes: named volumes persist GLPI data and MySQL. Backups should be taken before destructive operations.

Configuration & Secrets

- ``.env.example`` contains placeholders for variables like ``N8N_ENCRYPTION_KEY``, ``GRAFANA_ADMIN_PASSWORD``, and database passwords. It is safe to commit.
- Real secrets must only be placed in ``.env`` (git-ignored) or stored in secure credential stores. ``docs/10-security-and-secrets.md`` provides guidelines on rotation, backups, and access control.

Troubleshooting & Common Issues

- n8n errors: verify credential configuration and test individual nodes. Check n8n container logs for stack traces.
- GLPI connection problems: confirm GLPI container is running (``docker ps``) and API is reachable (``curl http://localhost:8080/apirest.php/initSession``).
- Loki/Prometheus issues: check endpoints (``http://localhost:3100/ready``, ``http://localhost:9090/targets``).
- Volume issues: failing containers or data loss may require restoring from backups — see ``docs/09-troubleshooting.md``.

Security Considerations

- No real credentials are committed. The repo includes a sanitized ``.env.example`` and documentation describing safe handling.

- n8n credential encryption requires `N8N\_ENCRYPTION\_KEY`; changing this key invalidates stored credentials unless migrated.
- For public sharing, owner-only notes and sensitive investigation artifacts were excluded (`_excluded-from-repo` / `.gitignore` updates).

Visuals & Screenshots

Screenshots used in documentation are in ``docs/screenshots/``. Files include:

- `agent-vocal-ia-interface.png`
- `docker-desktop-containers.png`
- `glpi-dashboard-overview.png`
- `grafana-docker-monitoring.png`
- `grafana-loki-logs-dashboard.png`
- `n8n-webhook-config.png`
- `n8n-workflow-diagram.png`

Recommendations & Next Steps

- For production: move from Docker Compose to orchestrator (Kubernetes) for scaling and resilience; use managed databases and secret management (HashiCorp Vault, cloud KMS).
- Harden GLPI and MySQL networking and backups; enable HTTPS with a reverse proxy.
- Add CI checks for docs, basic container linting, and a reproducible smoke test for workflows.
- Consider automated tests for n8n workflows (mocked API responses) and policy checks for exported workflows.

Appendix: Where to Look (key docs)

- ``docs/00-project-overview.md`` — project overview
- ``docs/01-architecture.md`` — architecture diagrams
- ``docs/02-installation.md`` — step-by-step install and start
- ``docs/03-configuration.md`` — environment variables and config
- ``docs/04-n8n-workflows.md`` — detailed workflow documentation
- ``docs/05-glpi-integration.md`` — GLPI API details
- ``docs/06-dashboard.md`` — dashboard / app details
- ``docs/07-monitoring-stack.md`` — Prometheus/Grafana/Loki details
- ``docs/08-docker-services.md`` — Docker Compose and services
- ``docs/09-troubleshooting.md`` — common fixes and checks
- ``docs/10-security-and-secrets.md`` — security guidance

Assumptions & TODOs

- Where exact runtime or production deployment choices are required (e.g., TLS certs, cloud provider), those are left as recommendations and will need a separate provisioning plan.

- If you want a more formal, templated report (PDF with cover page, table of contents, page numbers), I can produce a styled PDF and add page numbering and a cover.

End of report.